Set 5

Here are the instructions for each exercise in **Set 5**.

You can create a workspace for your **Set5** folder in **VS Code** by selecting **Open Folder...** and choosing your **Set5** folder. This allows you to access and manage all your exercise files in one place.

Open the **.c** file you want to run using **VS Code** or any other text editor you prefer (for example, Notepad on Windows). You can edit your C file directly in any of these editors.

When you finish writing your code, you need to **compile it**. To do this, open **w64devkit** and navigate to the **exercise folder** you are working on within your **Set5** directory. For detailed steps on how to navigate to the folder and compile your program, refer to the *"Getting started with LEU483"* guide on Canvas.

You don't need to do all exercises at once, some can be saved to practice for the exam.

1 Count words

Open: ex_word_count
Samples: sample_string

In this exercise you will perform simple manipulations of a string

Task:

- Start by just printing the string in the array `str`
- Count the number of words in the text and print the result
- Capitalize all letters in the text and print the resulting string

Program output:

```
This is a text with seven words.
Number of words: 7
THIS IS A TEXT WITH SEVEN WORDS.
```

2 Café

Open: ex_cafe/main.c
Samples: sample_string, sample_use_string_h

With your new knowledge of strings it's time to write a small program for a café. It should welcome the customer and ask for their order: what product and how many. The café serves the following:

- Coffee (45kr)
- Tea (40kr)
- Chocolate (50kr)
- Cinnamon bun (25kr)

Note that the program should work equally well for a loud customer (COFFEE) as for a quiet customer (coffee). And any other customer (Coffee, CoFFeE, etc.).

Tasks:

- Read the order from the customer using `fgets` and assign it to `input`.
- When reading the order with `fgets` a newline character will be included from hitting the enter key. Trim the input string such that it does not include the new line character.
- Make sure that the characters in the string are of the same case, this makes the next step easier.
- Implement the logic for calculating the price by comparing the input string to the four products defined above.
    - If the customer tries to order something that is not on the menu, the program should let customer know this, and then exit.
- Only after making sure that the item is on the menu should the program ask the customer for how many of the item they want.
- Finally, print the total cost of the order and exit.

Hints:
You may make use of the following functions

- `fgets`
- `strcmp`
- `strcspn`
- `tolower`

References

- Functions available in `string.h`: https://en.wikibooks.org/wiki/C_Programming/string.h

Example output:
Note that these are four separate runs of the program.

```
Welcome to the new and improved coffe shop.
Our new and improved system also accepts text as input.
First select the product and then the number of items.
Available products:
- Coffee (45kr)
- Tea (40kr)
- Chocolate (50kr)
- Cinnamon bun (25kr)
Enter order: Coffee
How many do you want: 2
Total cost: 90kr
Thank you!
```

```
Welcome to the new and improved coffe shop.
Our new and improved system also accepts text as input.
First select the product and then the number of items.
Available products:
- Coffee (45kr)
- Tea (40kr)
- Chocolate (50kr)
- Cinnamon bun (25kr)
Enter order: TEA
How many do you want: 1
Total cost: 40kr
Thank you!
```

```
Welcome to the new and improved coffe shop.
Our new and improved system also accepts text as input.
First select the product and then the number of items.
Available products:
- Coffee (45kr)
- Tea (40kr)
- Chocolate (50kr)
- Cinnamon bun (25kr)
Enter order: cinnamon BUN
How many do you want: 3
Total cost: 75kr
Thank you!
```

```
Welcome to the new and improved coffe shop.
Our new and improved system also accepts text as input.
First select the product and then the number of items.
Available products:
- Coffee (45kr)
- Tea (40kr)
- Chocolate (50kr)
- Cinnamon bun (25kr)
Enter order: Juice
Sorry, we don't have "juice"
Bye!
```

3 String functions

Open: ex_str_func/main.c
Samples: sample_string, sample_use_string_h, sample_string_func

In this exercise, you will practice writing functions for strings. You need to complete the body of the four functions that have already been declared.

Tasks:

- Write the function `count`. The function should count the number of characters in a string.
- Write the function `to_lower`. The function should convert a string to lowercase letters, and return this lowercase string. Only consider the English alphabet.
- Write the function `reverse`. This function should reverse a string. For example: `hello` should become `olleh`.
- Write the function `trim`. This function should remove any leading and trailing space in a string. For example: `' hello '` should become `'hello'`. You do not need to handle cases where spaces appear between characters within the string.
- The final output should print OK for all tests.

Hints:

- When converting an uppercase letter to a lowercase letter, you can make use of ASCII codes. For example, the difference between `'A'` and `'a'` is 32 (which is equivalent to the ASCII code for space, i.e. `' '`).

4 Product name

Open: ex_product_name/main.c

In this exercise you will write a function `getProductName` that finds and extract the product name, in this case "Orange" from a string that looks like this `Price: 1.99, Product: Orange, Weight: 0.2,`.

Note that there is also a second test string (`test_string2`) in the program for which the function should write "Kiwi" to `product_name`.

The function `getProductName` should be general enough to extract any product name from a string that follows the pattern of `test_string`. It should make use of the functions `strstr()` and `strchr()` from the `string.h` library.

Try adding additional test strings to make sure it works as expected.

Example output:

```
Orange
Kiwi
```

5 Robber language

Open: ex_robberlang/main.c
Samples: sample_use_string_h, sample_string_func

This exercise will encode strings in to the Rövarspråket (The Robber Language). To encode a string, every consonant should be doubled and an 'o' inserted in-between. Vowels are unchanged. See: https://en.wikipedia.org/wiki/R%C3%B6varspr%C3%A6ket

Task:

- Complete the function `to_robber`. The function takes a string `src` and applies the Robber Language to it, storing the encoded string in `dest`. In this version of the function, `dest` is allocated by callee, i.e. calling the function takes `dest` as an argument and changes `dest` in-place.
- Complete the function `to_robber2`. This function also takes a string `src`. However, the encoded string should be allocated dynamically.
- The final output should print OK for all tests.

Hints:

- Use functions from `string.h` as much as possible.

6 File

Open: ex_file/main.c

Implement a program to read and write text files.

Note: For this exercise, you need to read/write text files. Your code must be able to find these files. **Give the absolute path of your file when you open it**. Note also that Windows uses backslash in file paths and the character for backslash needs to be escaped using another backslash when writing strings in C. For example, to write the path 'C:\Users\username\somefolder\input_ex_file.txt' as a string in C you need to use `\\` as below:

```
char input_file[] = "C:\\Users\\username\\somefolder\\input_ex_file.txt";
```

Task:

- Read the content of the file 'input_ex_file.txt'. Print out the contents of the file in the terminal.
- Write the text stored in the array `text` in the file 'output_ex_file.txt'.

Example output:

```
This text is the file input_ex_file.txt
```

7 Crypto

Open: ex_crypto/main.c

In this exercise, you will encrypt and decrypt a text file. The text should be encrypted/decrypted using a user-inputted cipher key. This cipher key should be a small integer that is used to 'encode' every character in the file to a different character.

Task:

- Implement the function `encrypt`. The function should first open two files: 'input_ex_crypto.txt' (to read from) and 'out.txt' to write to. Then, the text in 'input_ex_crypto.txt' should be encoded and then written to 'out.txt', using the cipher key.
- Implement the function `decrypt`. This function should open two files: 'out.txt' (to read from) and 'out2.txt' (to write to). It should then decode the text in 'out.txt', using the cipher key.

Hints:

- If the cipher key entered is 1, it can be used to encode the character `'a'` to the character `'b'`.
- Try encoding 'input_ex_crypto.txt', writing the output to 'out.txt'. Then try decoding 'out.txt', writing the output to 'out2.txt'. Then, if you open 'out2.txt' (e.g. in Notepad on Windows), it should contain the same text as in 'input_ex_crypto.txt'.
- Open the files using absolute paths. In the example below the user (hanna) have placed the input and output files in the folder 'Programutveckling' on the C-drive on a Windows computer with absolute path 'C:\Users\hanna\Desktop\Programutveckling'. NOTE that the character for backslash needs to be escaped using another backslash.

Example output:

```
Select
1) Encrypt
2) Decrypt (-1 to quit)
> 1
Input (de)cipher key (small int) > 2
In file name >
C:\\Users\\hanna\\Desktop\\Programutveckling\\input_ex_crypto.txt
Out file name > C:\\Users\\hanna\\Desktop\\Programutveckling\\out.txt
Select
1) Encrypt
2) Decrypt (-1 to quit)
> 2
Input (de)cipher key (small int) > 2
In file name > C:\\Users\\hanna\\Desktop\\Programutveckling\\out.txt
Out file name > C:\\Users\\hanna\\Desktop\\Programutveckling\\out2.txt
```

8 Parenthesis match

Open: ex_paren_match/main.c
Samples: sample_string_func, sample_use_string_h

Check if a string has correctly matching parentheses, i.e. the same number of opening and closing parentheses and correct nesting.

Task:

- Write the function `index_of`. The function should return the index of a character in a string. If the character is not found, the function should return `-1`.
- Write the function `matches`, which should check that number of opening and closing parenthesis match and that they are in the correct place. It should return `true` if so.
- The final output should print `true` for all tests.

Hints:

- In the `matches` function, use a `char` array and an index. For each opening parenthesis, add it to the `char` array and increment the index. For each closing parenthesis, check that index contains the matching open parenthesis. If so, decrement the index and remove it (i.e. remove the matching pair). If the `char` array is empty when all parentheses are read, then everything matches okay.

9 Capitalize

Open: ex_capitalize/main.c

This is an old exam question. Try to solve this with pen and paper before using the computer.

Skriv en funktion, `first_cap`, som sätter första bokstaven i varje ord till att vara stor. Du kan anta att varje sträng börjar med ett ord och att alla andra ord föregås av ett mellanslag av något slag. Resultatet placeras i en annan sträng (`new_str`), enligt följande exempel:

Anrop	**new_str**
`first_cap(new_string, "some string");`	"Some String"
`first_cap(new_string, "one, two, three");`	"One, Two, Three"
`first_cap(new_string, "the Elephant\nis\tpink");`	"The Elephant\nIs\tPink"

Skapa funktionen och anropa den ifrån main för att testa den. Korrigerade strängar ska skrivas ut.

10 Pig latin

Open: ex_pig_latin/main.c

This is an old exam question. Try to solve this with pen and paper before using the computer.

Svinlatin är det svenska namnet på "pig latin", ett låtsasspråk som används av engelska barn. Gör färdigt programmet så det läser in ett ord och förvandlar det till svinlatin. För ord som börjar med konsonant, flyttas denna konsonant till slutet och "ay" läggs till. För ord som börjar med vokal, läggs "way" till. Se även https://sv.wikipedia.org/wiki/Pig_Latin, där du även hittar exempel.

Ord ska anges med enbart små bokstäver. Två körningar av programmet kan se ut så här:

```
Word to translate: horse
Pig latin: orsehay

Word to translate: owl
Pig latin: owlway
```

11 Dynamic allocation

Open: ex_dyn_allocate/main.c
Samples: sample_use_malloc

Implement a program that dynamically allocates an array of a size specified by the user (using `malloc`), and allow the user to interactively fill the resulting array with integer.

Example output:

```
How many elements > 3        (first run)
Input 0 >  1
Input 1 >  2
Input 2 >  3
Array is 1  2  3

How many elements > 5        (second run)
Input 0 >  7
Input 1 >  2
Input 2 >  4
Input 3 >  5
Input 4 >  1
Array is 7  2  4  5  1
```

12 Bitwise operators

Open: ex_bitwise/main.c

This is an exercise in using bitwise operators in C, it will be useful in upcoming courses. See also https://en.wikipedia.org/wiki/Bitwise_operations_in_C.

Task:

- Implement the function `byte_or`, which should apply the bitwise `or`-operator.
- Implement the function `byte_and`, which should apply the bitwise `and`-operator.
- Implement the function `byte_equals`, which should check whether two bytes are the same.
- Implement the function `byte_intersect`, which should check whether any bit is set in both bytes.
- Implement the function `byte_set_index`, which should set the bit at the specified index to 1.
- Implement the function `byte_get_index`, which should return whether the bit at the specified index is set or not.
- Implement the function `byte_clear_index`, which should set the bit at the specified index to 0.
- Uncomment all checks and the final output should print `ok` for all tests.

13 Remove duplicates dynamically

Open: ex_remdupl_dyn/main.c
Samples: sample_use_malloc

This is a rework of exercise 12 (Remove duplicates) from set 4, this time by implementing a dynamically allocated array instead of padding with zeros.

Task:

- Implement the function `remove_dupl` that removes duplicates from a **sorted** array. The returned array is being allocated dynamically and only contains values that are larger than 0.
- The final output should print `ok` for all tests.

Hints:

- Use a temporary variable (`tmp`) with same size as `arr` and copy the unique values to `tmp`.
- Keep track of the number of elements in `tmp`.
- Then, dynamically allocate an array of that size and copy values from `tmp` to the allocated array.
- Return the allocated array with the correct size.